

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«РОССИЙСКАЯ АКАДЕМИЯ НАРОДНОГО ХОЗЯЙСТВА И
ГОСУДАРСТВЕННОЙ СЛУЖБЫ
при ПРЕЗИДЕНТЕ РОССИЙСКОЙ ФЕДЕРАЦИИ»

НИЖЕГОРОДСКИЙ ИНСТИТУТ УПРАВЛЕНИЯ - ФИЛИАЛ

Факультет Управления

Кафедра информатики и информационных технологий

Направление подготовки / Специальность 09.03.03 Прикладная информатика

Образовательная программа Корпоративные информационные системы управления

ОТЧЕТ

о прохождении

практики технологической (проектно-технологической)

(вид практики)

Соколова Дмитрия Александровича

(Ф.И.О. обучающегося)

3 курс обучения

учебная группа № Ик-731

Место прохождения практики ООО «САТЕЛ ПрО», Отдел проектных разработок

Департамента унифицированных коммуникаций Основного подразделения,

г. Москва, ул. Щепкина, д. 28

(указывается полное наименование структурного подразделения Института / профильной организации и ее структурного подразделения, а также их фактический адрес)

Срок прохождения практики: с « 27 » апреля 2026 г. по « 26 » мая 2026 г.

Руководители по практической подготовке:

От института: Н.М. Трубилов, заместитель директора библиотечно-ресурсного центра

(И.О. Фамилия, должность)

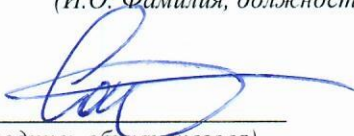
От профильной организации

(при наличии):

С.М. Митричев, руководитель отдела разработки

(И.О. Фамилия, должность)

Отчет подготовлен


(подпись обучающегося)

Д.А. Соколов

(И.О. Фамилия)

г. Нижний Новгород , 2026 г.

Оглавление

Введение	4
1 Характеристика профильной организации ООО «САТЕЛ»	6
1.1 Общая характеристика организации	6
1.2 Основные направления деятельности ООО «САТЕЛ»	6
1.3 Программные продукты и решения организации	6
1.4 Характеристика платформы РТУ как объекта разработки	7
1.5 Организация рабочего процесса frontend-разработчика	8
2 Теоретические и методические основы выполнения практики	9
2.1 Постановка индивидуального задания и цели практики	9
2.2 Анализ литературных и технических источников по теме практики	9
2.3 Методы исследования, анализа и выполнения работ	10
2.4 Информационные технологии в профессиональной деятельности frontend-разработчика	12
2.5 Программные продукты и инструменты, применяемые при выполнении задач	14
2.6 Требования к оформлению технической и проектной документации	16
2.7 Порядок внедрения результатов разработки в рабочий программный продукт	19
3 Практическая часть. Выполнение задач в ООО «САТЕЛ»	22
3.1 Получение и анализ практических задач	22
3.2 Доработка компонентов общей UI-библиотеки	24
3.2.1 Приведение компонента radio-button в соответствие с дизайн- системой	24
3.2.2 Обновление компонента Wui-input-switch по макету	25
3.2.3 Обновление демонстрационных страниц и Storybook	27
3.2.4 Написание и актуализация unit-тестов UI-компонентов	27

3.3	Доработка конфигурации запуска проекта в dev-режиме	28
3.3.1	Анализ проблемы хардкода backend-адресов	28
3.3.2	Вынесение параметров окружения в env-файлы	30
3.3.3	Синхронизация формирования URL в dev-server и runtime	30
3.3.4	Исправление формы авторизации	32
3.4	Реализация функциональности аппаратной индикации	33
3.4.1	Анализ сценария работы аппаратной кнопки микрофона	34
3.4.2	Реализация глобального состояния goose-модулей	34
3.4.3	Синхронизация UI-кнопки mute и аппаратной индикации	36
3.4.4	Особенности режимов stateful и push-to-talk	38
	Заключение	39
	Библиографический список	40

Введение

Проектно-технологическая практика была пройдена в период с 27 апреля по 26 мая 2026 года в ООО «САТЕЛ» на должности младшего frontend-разработчика. Компания занимается разработкой программных и аппаратных решений в области унифицированных коммуникаций, корпоративной телефонии и специализированных пользовательских интерфейсов. В период практики основное внимание было уделено участию в разработке frontend-части программного продукта «Дилинговый пульт РТУ».

Целью прохождения практики является закрепление и развитие профессиональных знаний, умений и навыков в области frontend-разработки.

Для достижения указанной цели были поставлены следующие задачи практики:

1. изучить деятельность ООО «САТЕЛ», профиль организации и назначение разрабатываемого программного продукта;
2. ознакомиться с архитектурой frontend-проекта, технологическим стеком и внутренним процессом разработки;
3. проанализировать требования к практическим задачам, макеты интерфейса и техническую документацию проекта;
4. принять участие в доработке компонентов пользовательского интерфейса, конфигурации dev-режима и логики аппаратной индикации;
5. закрепить навыки тестирования, оформления результатов разработки и работы с инструментами командной разработки.

Объектом практики является процесс разработки и сопровождения frontend-части программного продукта ООО «САТЕЛ». Предметом практики являются методы и инструменты создания пользовательского интерфейса.

В ходе практики использовались современные технологии и инструменты frontend-разработки: Vue 3, TypeScript, Vite, Pinia, Tailwind CSS, Storybook, Git, GitLab, Jira, Figma, а также средства локального тестирования и отладки [12–22]. Работа выполнялась в рамках командного процесса: от анализа задачи и изучения требований до реализации, проверки, оформления результата и передачи изменений на ревью.

В результате практики были закреплены навыки анализа требований, работы с кодовой базой, доработки UI-компонентов, взаимодействия с системой контроля версий и участия в общем процессе разработки программного обеспечения.

1 Характеристика профильной организации ООО «САТЕЛ»

1.1 Общая характеристика организации

ООО «САТЕЛ» - российская компания, работающая в сфере цифровых технологий, разработки программного обеспечения, телекоммуникационных решений, производства оборудования и системной интеграции. Согласно официальной информации, компания работает на российском рынке цифровых технологий с 1995 года и за время деятельности расширила спектр предлагаемых продуктов и услуг [10].

Профиль деятельности ООО «САТЕЛ» связан с созданием и внедрением решений для корпоративных коммуникаций, промышленной автоматизации.

В период прохождения практики моя деятельность была связана с frontend-разработкой программных продуктов компании.

1.2 Основные направления деятельности ООО «САТЕЛ»

Основные направления деятельности ООО «САТЕЛ» можно разделить на несколько групп: программные решения, производство оборудования, инжиниринг и выполнение отраслевых проектов. На официальном сайте компании представлены направления, связанные с корпоративными коммуникациями, промышленными решениями, ИТ-инфраструктурой, автоматизацией производства, комплексной безопасностью, сервисом и аутсорсингом [10].

1.3 Программные продукты и решения организации

В продуктовую линейку ООО «САТЕЛ» входят решения для корпоративных коммуникаций и промышленных задач. На официальном сайте среди программных продуктов указаны РТУ, РТУ-Коннект, РТУ-Атмосфера, ZIAX, СИЗАР, СОБА, VizorLabs, СМАРТ-решения, Энербас и другие продукты [10].

Ключевым направлением, связанным с прохождением практики, является платформа РТУ. РТУ используется для построения распределенных и локальных сетей унифицированных коммуникаций. Платформа включает VoIP-функциональность, модули диспетчерской и селекторной связи, корпоративный мессенджер, видеоконференцсвязь и клиентские приложения [9].

1.4 Характеристика платформы РТУ как объекта разработки

РТУ (Российский телефонный узел) – многокомпонентное решение для построения распределенных и локальных сетей унифицированных коммуникаций.

Представляет собой VoIP-платформу операторского класса надежности, в которой дополнительно реализованы модули диспетчерской и селекторной связи, корпоративный мессенджер, видеоконференцсвязь, а также собственное мобильное приложение для iOS/Android [9].

Решение позволяет внедрять сервисы унифицированных коммуникаций как в существующую инфраструктуру, так и получать «из облака».

В рамках практики рассматривался один из frontend-продуктов экосистемы РТУ - «Дилинговый пульт РТУ». Это кроссплатформенное абонентское приложение для речевого дилинга, предназначенное для управления телефонными звонками, конференциями и коммуникациями в банковской и биржевой торговле [2]. Основными пользователями такого продукта являются трейдеры, брокеры и финансовые специалисты, которым требуется быстро обрабатывать несколько параллельных голосовых сессий.

Особенность дилингового пульта заключается в поддержке «завешенных» линий. В отличие от классической телефонии, где пользователь обычно работает с одним активным вызовом, дилинговый пульт позволяет одновременно слушать и обслуживать несколько активных линий. Согласно проектной спецификации, приложение поддерживает до 16

активных завешенных вызовов одновременно [2]. Также дилинговый пульт может содержать множество аудиомодулей. Помимо пользовательского интерфейса прибор оснащен кнопками с индикацией.

1.5 Организация рабочего процесса frontend-разработчика

Рабочий процесс frontend-разработчика в ООО «САТЕЛ» строится по этапам, характерным для промышленной командной разработки: анализ задачи, разработка, локальная проверка, оформление merge request, code review, тестирование и включение изменений в релизную ветку [1].

Работа над задачей начинается со стадии анализа. На этом этапе необходимо изучить описание задачи в Jira, комментарии коллег, связанные задачи, макеты Figma, технические материалы и требования к версии, в которую должно попасть изменение [22]. Если информации недостаточно, задача переводится в запрос дополнительной информации. После завершения анализа определяется трудоемкость, после чего задача переходит в разработку.

На стадии разработки создается отдельная ветка в системе контроля версий. Название ветки обычно связано с типом задачи и ее идентификатором в Jira. Перед началом работы разработчик обновляет актуальную ветку, вносит изменения в код, выполняет локальную проверку, запускает линтер и тесты. После завершения разработки создается merge request, в котором указывается, что было сделано и как проверить результат [13; 21].

После code review задача передается на тестирование. Далее изменения попадают в develop-ветку, затем проходят тестовый стенд и включаются в релизную сборку.

2 Теоретические и методические основы выполнения практики

2.1 Постановка индивидуального задания и цели практики

В соответствии с индивидуальным заданием проектно-технологическая практика была направлена на закрепление теоретических знаний и получение практического опыта в профессиональной деятельности frontend-разработчика. Основное содержание задания включало изучение литературных и технических источников по теме разрабатываемого проекта, анализ методов выполнения работ, применение информационных технологий, изучение программных продуктов профессиональной сферы, требований к оформлению технической документации и порядка внедрения результатов разработки.

Целью практики являлось развитие профессиональных компетенций в области frontend-разработки путем участия в сопровождении и доработке программного продукта ООО «САТЕЛ» - дилингового пульта РТУ.

Для достижения цели были поставлены следующие задачи:

1. изучить деятельность организации и назначение разрабатываемого программного продукта;
2. ознакомиться с архитектурой frontend-проекта, технологическим стеком и внутренними процессами разработки;
3. проанализировать требования к задачам, макеты интерфейса и техническую документацию проекта;
4. принять участие в доработке компонентов пользовательского интерфейса и логики приложения;
5. изучить порядок тестирования, документирования и внедрения результатов разработки.

2.2 Анализ литературных и технических источников по теме практики

При выполнении проектно-технологической практики были изучены литературные и технические источники, связанные с frontend-разработкой,

проектированием пользовательских интерфейсов, архитектурой web-приложений и средствами командной разработки.

Учебная литература

К основным теоретическим источникам относятся учебные материалы по web-программированию, JavaScript, TypeScript, разработке пользовательских интерфейсов, тестированию программного обеспечения и оформлению технической документации [3-8].

Официальная документация используемых инструментов

Официальная документация Vue 3, TypeScript, Vite, Pinia, Vue Router, Storybook, Git, GitLab, Figma и Electron использовалась для уточнения особенностей работы с компонентами, состоянием приложения, маршрутизацией, сборкой проекта, тестированием и системой контроля версий [11–23].

Внутренние документы организации

Также были изучены внутренние документы ООО «САТЕЛ»: спецификация проекта «Дилинговый пульт РТУ», документация по frontend-онбордингу, описания задач в Jira, макеты интерфейса в Figma и комментарии к выполненным задачам [1; 2; 12; 22]. Эти источники позволили понять назначение продукта, его архитектуру, основные пользовательские сценарии, порядок работы с задачами и требования к качеству кода.

2.3 Методы исследования, анализа и выполнения работ

При выполнении проектно-технологической практики использовались следующие методы исследования, анализа и выполнения работ, которые являются обязательными при работе в ООО «САТЕЛ»:

1. Анализ требований задачи

На начальном этапе изучалось описание задачи в Jira, ее тип, статус, планируемая версия исправления, комментарии коллег и связанные материалы [22].

2. Анализ проектной документации

Для понимания логики продукта изучалась спецификация проекта «Дилинговый пульт РТУ», внутренняя документация по frontend-разработке [1; 2].

3. Анализ дизайн-макетов

При доработке компонентов пользовательского интерфейса использовались макеты в Figma [12].

4. Сравнительный анализ текущей и требуемой реализации

Перед внесением изменений сравнивалось фактическое поведение компонента или модуля с требуемым.

5. Декомпозиция задачи

Сложные задачи разбивались на отдельные этапы (или отдельные задачи): анализ текущего кода, поиск связанных модулей, внесение изменений, локальная проверка, обновление тестов и оформление результата.

6. Метод компонентного подхода

При работе с frontend-частью применялся компонентный подход. Интерфейс рассматривался как набор переиспользуемых компонентов, каждый из которых имеет входные параметры, события, состояния. Также необходимо было соблюдать подход FSD, который четко разграничивает расположение модулей, компонентов и функций.

7. Статический анализ кода

В процессе разработки проверялась структура кода, типы данных, зависимости между модулями, использование переменных окружения, корректность импортов и соответствие принятому стилю проекта.

8. Локальное тестирование

После внесения изменений выполнялась проверка проекта в локальной среде. Проверялись основные пользовательские сценарии.

9. Модульное и компонентное тестирование

Для проверки корректности отдельных функций и компонентов использовались unit-тесты и компонентные тесты. Они позволяли убедиться, что компонент корректно отображает данные, реагирует на действия пользователя и передает события [5; 19].

10. Документирование результатов работы

После завершения задачи результат фиксировался в комментарии к задаче в Jira или merge request в GitLab. Указывалось, что было изменено, какие файлы затронуты, какие сценарии проверены и как можно протестировать выполненную доработку [21; 22].

2.4 Информационные технологии в профессиональной деятельности frontend-разработчика

В профессиональной деятельности frontend-разработчика информационные технологии применяются на всех этапах создания и сопровождения программного продукта: от анализа задачи и проектирования интерфейса до реализации, тестирования и передачи изменений в релиз.

Основными информационными технологиями, применяемыми в работе, являются:

1. Технологии разработки пользовательского интерфейса

Для создания интерфейса используются Vue 3, TypeScript, Tailwind CSS и компонентный подход [16; 17; 20]. Интерфейс строится из отдельных компонентов, которые могут переиспользоваться в разных частях приложения. Такой подход применялся при доработке компонентов общей UI-библиотеки, например radio-button и Wui-input-switch.

2. Технологии управления состоянием приложения

Для управления состоянием используется Pinia [14]. Состояние приложения включает данные о пользователе, активных звонках, завешенных линиях, микрофоне, аудиоустройствах, настройках и других элементах системы.

3. Технологии сборки и настройки окружений

Для сборки и запуска проекта используется Vite [18]. В процессе практики выполнялась доработка конфигурации dev-режима: адреса backend-серверов были вынесены в env-файлы, а логика формирования URL была синхронизирована между dev-server и runtime.

4. Технологии взаимодействия frontend с backend

Frontend-приложение взаимодействует с backend-сервисами через API и WebSocket-соединения. Через API выполняются операции авторизации, получения данных пользователя, контактов, настроек и других сущностей. WebSocket используется для сценариев, где требуется обмен данными в реальном времени.

5. Технологии WebRTC и SIP

Так как продукт связан с телефонией, важную роль играют WebRTC и SIP. WebRTC используется для передачи аудио в браузере или desktop-приложении, а SIP - для сигнализации и управления телефонными сессиями [9].

6. Технологии интеграции с аппаратными контроллерами

Особенностью проекта является работа с аппаратными устройствами: Goose, Handset и Hub. Они используются для управления микрофоном, трубками, кнопками и световой индикацией. Обмен с контроллером выполняется через локальное WebSocket-соединение [9].

7. Технологии тестирования

Для проверки корректности изменений используются unit-тесты, компонентные тесты и ручное тестирование [5; 19]. При доработке UI-компонентов проверялись рендер текста, передача событий, disabled-

состояния, расположение элементов и корректность отображения разных вариантов компонента.

Также применялось ручное тестирование пользовательских сценариев: запуск проекта, проверка интерфейса, проверка состояний кнопок, поведения форм, работы авторизации и переключения режимов.

8. Технологии командной разработки

В работе применяются Git, GitLab, Jira, Figma и Storybook [12; 13; 15; 21; 22]. Jira используется для постановки и отслеживания задач, GitLab - для хранения кода, создания веток и merge request, Figma - для анализа макетов, Storybook - демонстрации UI-компонентов.

Таким образом, информационные технологии в деятельности frontend-разработчика используются комплексно.

2.5 Программные продукты и инструменты, применяемые при выполнении задач

При выполнении задач проектно-технологической практики использовались программные продукты и инструменты, применяемые в профессиональной frontend-разработке.

1. Vue 3

Vue - это JavaScript-фреймворк для создания пользовательских интерфейсов. Согласно официальной документации, Vue является производительным и универсальным фреймворком для разработки web-интерфейсов. В проекте Vue 3 использовался для создания компонентов, страниц и пользовательских сценариев дилингового пульта [20].

2. TypeScript

TypeScript - это строго типизированный язык программирования, основанный на JavaScript. Он добавляет к JavaScript систему типов и улучшает инструменты разработки. В проекте TypeScript применялся для повышения надежности кода, описания типов данных, интерфейсов и структур событий [17].

3. Vite

Vite - это инструмент сборки, предназначенный для быстрой разработки современных web-проектов. Он включает dev-сервер и механизм быстрой пересборки при изменении кода. В рамках практики Vite использовался для запуска проекта, настройки dev-режима и работы с переменными окружения [18].

4. Pinia

Pinia - библиотека для управления состоянием Vue-приложений. В официальной документации store описывается как сущность, хранящая состояние и бизнес-логику вне дерева компонентов. В проекте Pinia применялась для хранения состояния звонков, устройств, пользователя, настроек и завешенных линий [14].

5. Tailwind CSS

Tailwind CSS использовался для стилизации интерфейса. С его помощью задавались размеры, отступы, цвета, состояния компонентов и адаптация элементов интерфейса под дизайн-систему проекта [16].

6. Storybook

Storybook - инструмент для разработки, тестирования и документирования UI-компонентов в изоляции. В проекте он применялся для проверки компонентов общей библиотеки, например переключателей, кнопок и других элементов интерфейса [15].

7. Vitest

Vitest - тестовый фреймворк, созданный для работы с Vite-проектами. Он понимает конфигурацию Vite и использует те же механизмы обработки модулей. В рамках практики Vitest применялся для проверки корректности работы компонентов и отдельных функций [19].

8. Git и GitLab

Git - распределенная система контроля версий, предназначенная для отслеживания изменений в файлах проекта. GitLab использовался как

платформа для хранения репозитория, работы с ветками, merge request и code review [13; 21].

9. Jira

Jira применялась для постановки, описания и отслеживания задач. В ней фиксировались требования, статусы задач, комментарии, связи с commit и merge request, а также результаты анализа и разработки [22].

10. Figma

Figma использовалась для просмотра дизайн-макетов и UI Kit. Согласно официальному описанию, Figma является совместной платформой для проектирования интерфейсов и разработки продуктов. В рамках практики она применялась при доработке UI-компонентов в соответствии с дизайн-системой [12].

11. Electron

Electron - фреймворк для создания desktop-приложений с использованием JavaScript, HTML и CSS. Он позволяет поддерживать единую кодовую базу и создавать приложения для Windows, macOS и Linux. В проекте Electron применялся для desktop-версии диллингового пульта [11].

Таким образом, используемые инструменты обеспечивали полный цикл frontend-разработки: анализ требований, реализацию интерфейса, управление состоянием, сборку проекта, тестирование, работу с системой контроля версий и передачу изменений на проверку.

2.6 Требования к оформлению технической и проектной документации

В ООО «САТЕЛ» оформление технической и проектной документации является обязательным требованием. Материалы и документы, используемые в работе: задачи в Jira, проектная документация, макеты интерфейса, merge request, комментарии к выполненным задачам и результаты тестирования.

1. Документация задачи в Jira

К оформлению задачи предъявляются следующие требования [22]:

1. указание типа задачи: новая функциональность, ошибка, техническая работа;
2. указание статуса задачи;
3. указание приоритета;
4. указание версии продукта, в которую должно попасть исправление;
5. указание компонента продукта;
6. наличие описания проблемы или требуемой доработки;
7. наличие ссылок на связанные задачи, макеты, спецификации или обращения заказчика;
8. фиксация комментариев по результатам анализа и разработки.

2. Проектная документация

Проектная документация должна содержать [2]:

1. назначение программного продукта;
2. основные пользовательские сценарии;
3. функциональные требования;
4. описание архитектуры приложения;
5. описание технологического стека;
6. перечень известных ограничений и рисков;
7. информацию о развертывании и поддержке продукта.

3. Документация по дизайну и интерфейсу

Документация по интерфейсу оформляется на основе макетов и дизайн-системы [12]. Она должна включать:

1. описание визуальных состояний компонентов;
2. размеры, отступы и расположение элементов;
3. состояния активности, неактивности и disabled;
4. варианты отображения компонента;
5. описание поведения компонента при взаимодействии пользователя;
6. соответствие компонентов макетам Figma и UI Kit.

4. Документация UI-компонентов в Storybook

Для компонентов общей библиотеки необходимо [15]:

1. поддерживать актуальные демонстрационные примеры;
2. отображать основные состояния компонента;
3. показывать варианты размеров и расположения;
4. описывать входные параметры и события компонента;
5. обновлять stories при изменении внешнего вида или поведения;
6. обеспечивать возможность проверки компонента изолированно от основного приложения.

5. Документация изменений в GitLab и merge request

При оформлении merge request необходимо [21]:

1. указывать название, соответствующее задаче в Jira;
2. кратко описывать выполненные изменения;
3. указывать способ проверки результата;
4. прикладывать скриншоты при изменении интерфейса;
5. связывать merge request с задачей;
6. не включать в изменения временный, закомментированный или отладочный код;
7. проверять актуальность ветки перед передачей на ревью.

6. Требования к оформлению кода

Код также является частью технической документации проекта, поэтому к нему предъявляются требования [17; 20]:

1. использование TypeScript для явного описания типов;
2. соблюдение архитектуры Feature-Sliced Design;
3. распределение файлов по слоям проекта;
4. использование Vue 3 Composition API;
5. соблюдение правил ESLint;
6. отсутствие неиспользуемых импортов и переменных;
7. отсутствие лишних комментариев и временных решений;
8. понятные названия переменных, функций и компонентов.

7. Требования к безопасности технической информации

В рабочих материалах не допускается указывать:

1. пароли;
2. токены;
3. приватные ключи;
4. внутренние адреса серверов без необходимости;
5. конфиденциальные данные заказчиков;
6. персональные данные пользователей;
7. закрытую информацию, не относящуюся к выполнению задачи.

2.7 Порядок внедрения результатов разработки в рабочий программный продукт

Внедрение результатов разработки в ООО «САТЕЛ» происходит в несколько этапов. Инструменты затронутые при внедрении результатов разработки: Jira, GitLab, система контроля версий Git, CI/CD-пайплайны и тестовые стенды [1; 13; 21; 22].

Краткий инструктаж по внедрению описан в онбординге, а также использовался на практике:

1. Анализ задачи

Работа начинается с изучения задачи в Jira [22]. На этом этапе необходимо проанализировать описание, комментарии коллег, прикрепленные материалы, макеты Figma, технические спецификации и связанные задачи. Также определяется тип задачи: новая функциональность (feature), исправление ошибки (bugfix) или техническая работа (service-task).

2. Завершение анализа

После изучения задачи она переводится в статус «Анализ завершён». На этом этапе оценивается трудоемкость, фиксируется затраченное время и при необходимости указываются предполагаемые сроки выполнения. Если данных недостаточно, задача переводится в запрос информации (например

запросить информацию у backend-разработчиков, дизайнеров, тестировщиков).

3. Разработка

После перехода задачи в разработку создается отдельная ветка в Git [13; 21]. Название ветки формируется с учетом типа задачи и ее идентификатора в Jira, например feature/WUI-2405. Перед началом работы необходимо обновить актуальную ветки develop, master и hotfix. Затем отвести ветку по задаче, после чего вносит изменения в код.

4. Локальная проверка

После реализации изменений выполняется локальная проверка. Она включает в себя: запуск проекта, проверка основных пользовательских сценариев, пустых состояний, ошибок сервера и корректность отображения интерфейса. Также запускаются линтер и тесты командами `npm run lint` (форматирование) и `npm run test` (типы TypeScript) [19; 21].

5. Создание merge request

После локальной проверки создается merge request в GitLab [21]. В нем указывается название, соответствующее задаче в Jira, краткое описание выполненных изменений, способ проверки результата и, при необходимости, скриншоты для UI-доработок. Для проверки назначаются ревьюверы.

6. Code review

На этапе code review другие разработчики проверяют качество реализации. При наличии замечаний разработчик вносит правки и повторно отправляет merge request на проверку.

7. Тестирование

После прохождения ревью задача передается на тестирование. QA-специалист проверяет корректность выполненной задачи. Если в ходе проверки обнаруживаются ошибки, задача возвращается в разработку или создаются дополнительные подзадачи.

8. Попадание изменений в develop и релиз

После успешного прохождения ревью и тестирования изменения попадают в ветку `develop`. Далее код уходит на тестовый стенд, где проходит дополнительную проверку. Для подготовки релиза используется ветка `release/*`, а стабильная версия продукта хранится в ветке `master`.

9. Деплой

Деплой выполняется через GitLab CI/CD [21]. После подготовки релизного кандидата выполняется тегирование версии, после чего развертывание в `production` может выполняться автоматически или вручную через CI/CD-пайплайн. Для срочных исправлений используется отдельная `hotfix`-ветка от стабильной версии, после чего изменения возвращаются обратно в `develop`. На практике было применено ручное развертывание.

Таким образом, внедрение результатов разработки происходит поэтапно: задача проходит анализ, реализацию, локальную проверку, `code review`, тестирование, включение в `develop`, подготовку релиза и деплой. Такой порядок снижает риск ошибок и обеспечивает стабильность рабочего программного продукта.

3 Практическая часть. Выполнение задач в ООО «САТЕЛ»

3.1 Получение и анализ практических задач

Практические задачи выполнялись в рамках проекта «Дилинговый пульт РТУ», задачи поступали через систему Jira [22]. Процесс разработки и внедрения соответствовал описанному во второй главе.

В ходе практики было выполнено большое количество задач связанных с исправлением ошибок, доработкой программного продукта и добавлением нового функционала.

Для демонстрации были выбраны одни из основных и крупных задач, которые приведены в таблице 3.1 [22].

Таблица 3.1

Практические задачи, выполненные в период практики

Номер задачи	Направление	Краткое содержание
WUI-4159	UI-библиотека	Приведение компонента radio-button в соответствие с дизайн-системой
WUI-4218	UI-библиотека	Обновление компонента Wui-input-switch по макету
WUI-4207	Конфигурация проекта	Исправление конфигурации запуска в dev-режиме
WUI-4278	Дилинговый пульт	Реализация световой индикации аппаратной кнопки микрофона
WUI-3798	Дилинговый пульт UI	Модификация вкладки «Все контакты» для отображения выборочных контактов
WUI-4279	Дилинговый пульт UI	Вывод клавиатуры dialpad на видное место
WUI-4280	Дилинговый пульт UI	Улучшение визуальной подсветки карточки с активным вызовом

На рисунках 3.1-3.2 продемонстрирована Agile-доска с реальными задачами, которые выполнялись в ходе практики и их статусами. Данные задачи затрагивают 2 проекта: «Дилинговый пульт РТУ» и «UI библиотека».

На рисунках скрыта часть информации, по согласованию с руководителем практики от предприятия.

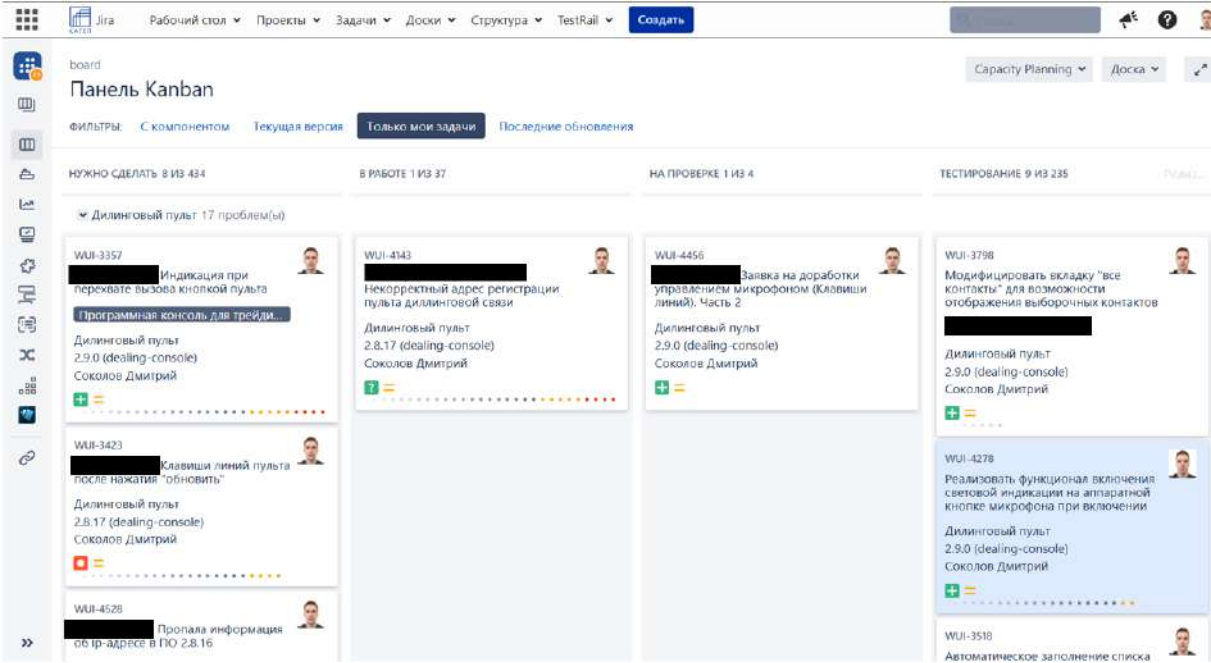


Рис. 3.1. Список задач проекта «Дилинговый пульт РТУ»

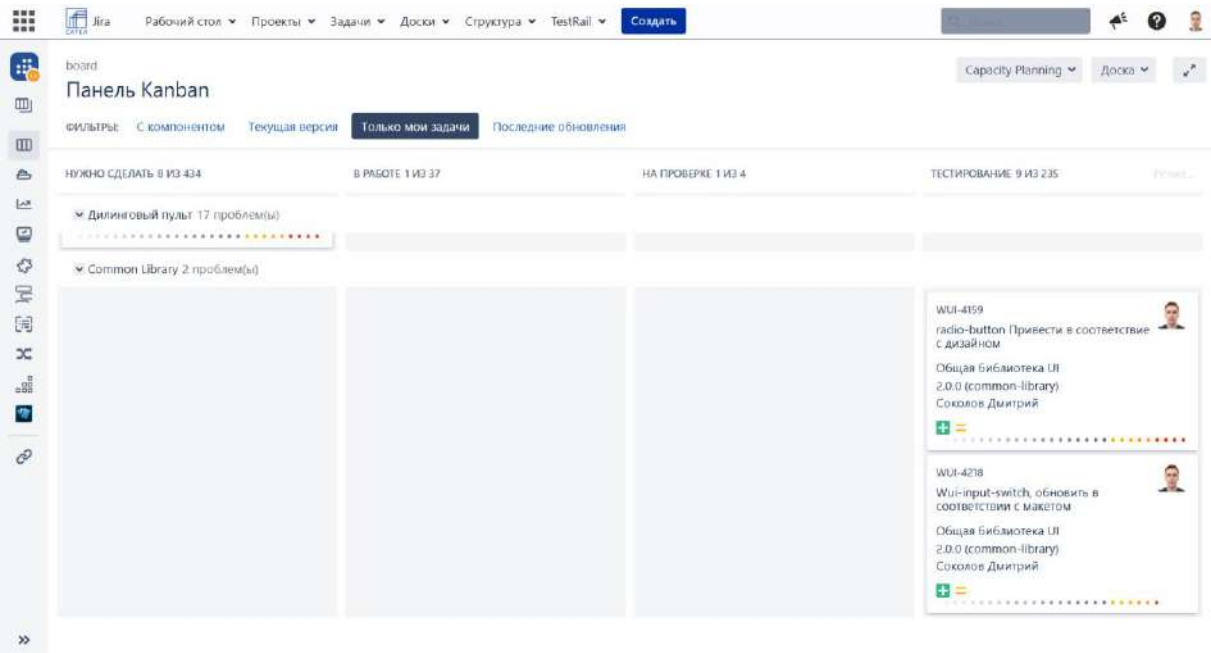


Рис. 3.2. Список задач проекта «UI библиотека»

3.2 Доработка компонентов общей UI-библиотеки

Одним из направлений практической работы стала доработка компонентов общей UI-библиотеки. Такие компоненты используются в разных продуктах компании, поэтому они должны быть единообразными, соответствовать дизайн-системе и корректно работать во всех предусмотренных состояниях.

3.2.1 Приведение компонента radio-button в соответствие с дизайн-системой

В рамках задачи WUI-4159 требовалось привести компонент radio-button в соответствие с актуальным дизайном. Основной целью было обеспечить единый внешний вид компонента в общей UI-библиотеке.

На этапе анализа был изучен макет компонента в Figma и текущая реализация в проекте [12; 22]. Особое внимание уделялось визуальным состояниям: выбранное состояние, невыбранное состояние, disabled-состояние, а также соответствие размеров и отступов дизайн-системе.

В ходе выполнения задачи были проверены стили компонента, логика отображения состояния выбора и поведение при взаимодействии пользователя. Доработка такого компонента важна, так как radio-button относится к базовым элементам интерфейса и может использоваться в настройках, формах и других частях приложения.

Макет в Figma и отчет по задаче в Jira приведен на рисунках 3.3-3.4 [12; 22].

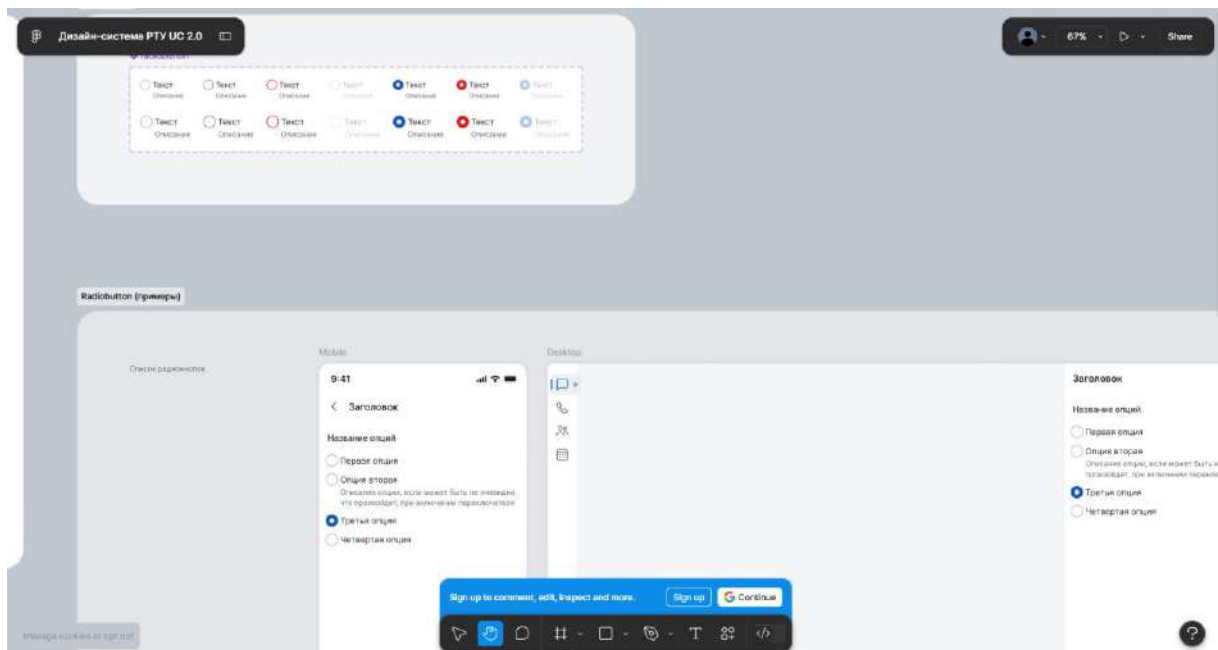


Рис. 3.3. Макет в Figma по задаче WUI-4159

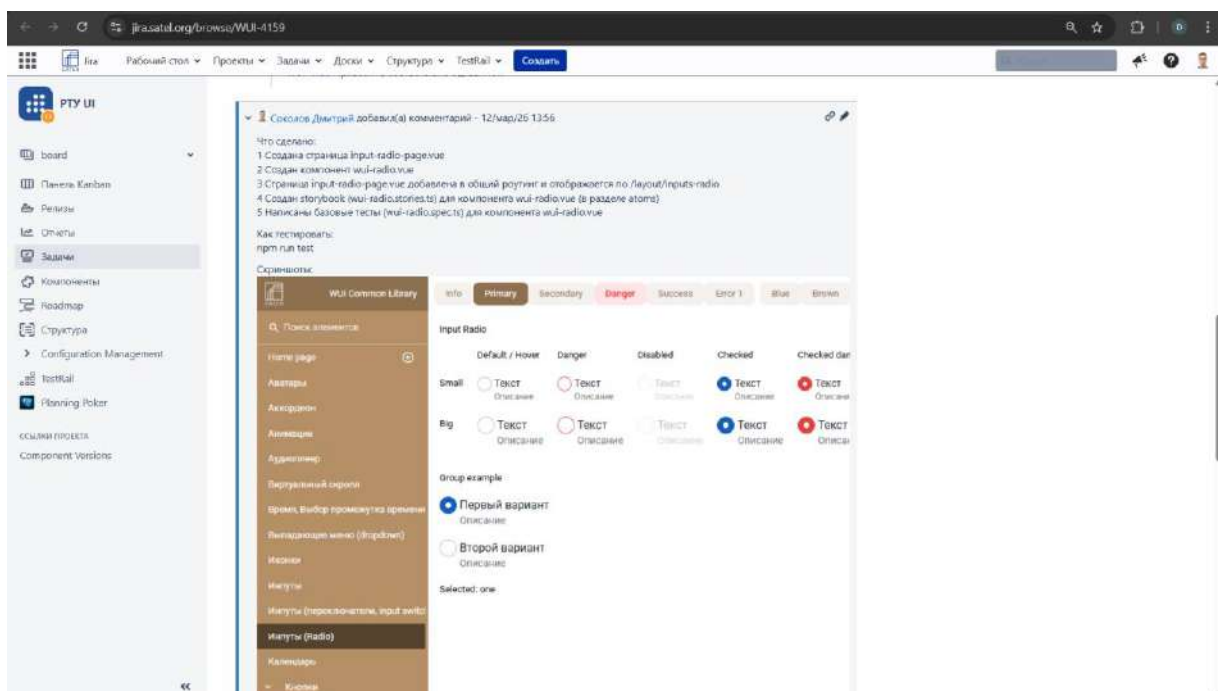


Рис. 3.4. Отчет в Jira по задаче WUI-4159

3.2.2 Обновление компонента Wui-input-switch по макету

Следующей задачей была доработка компонента Wui-input-switch в рамках задачи WUI-4218.

В процессе работы компонент был обновлен в соответствии с новым макетом [12; 22]. Были реализованы состояния on/off, disabled, размеры small/big, отображение label и description, а также возможность расположения переключателя слева или справа через параметр right.

Результат работы и макет в Figma приведены на рисунках 3.5-3.6.

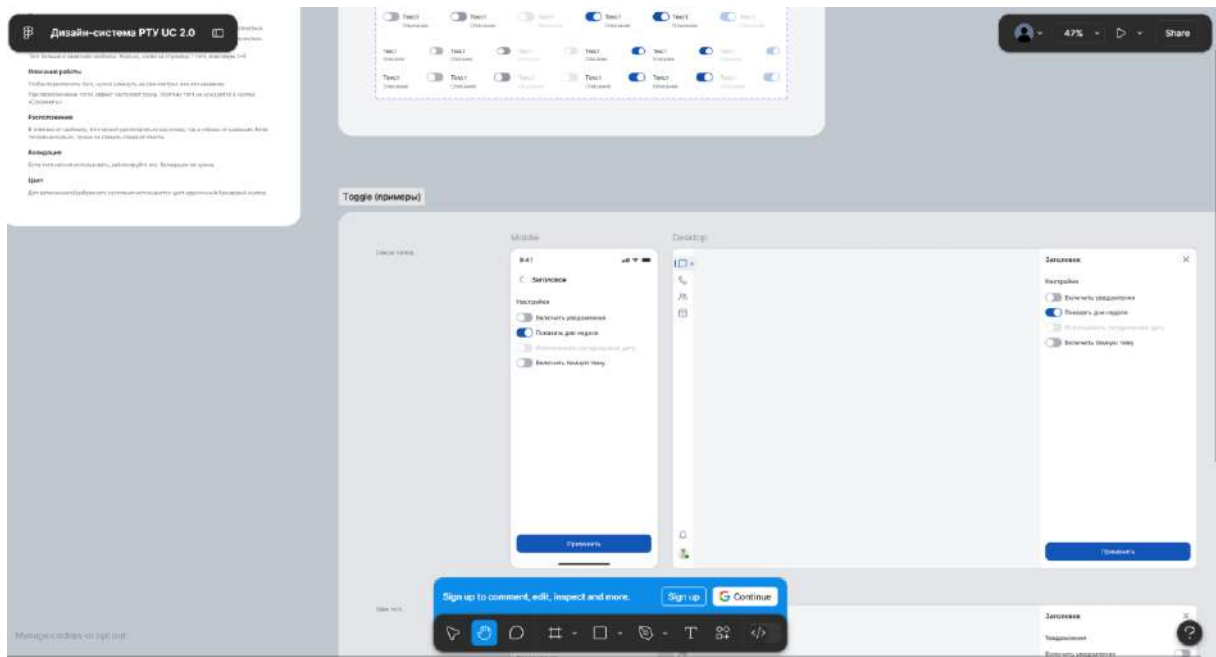


Рис. 3.5. Макет в Figma по задаче WUI-4218

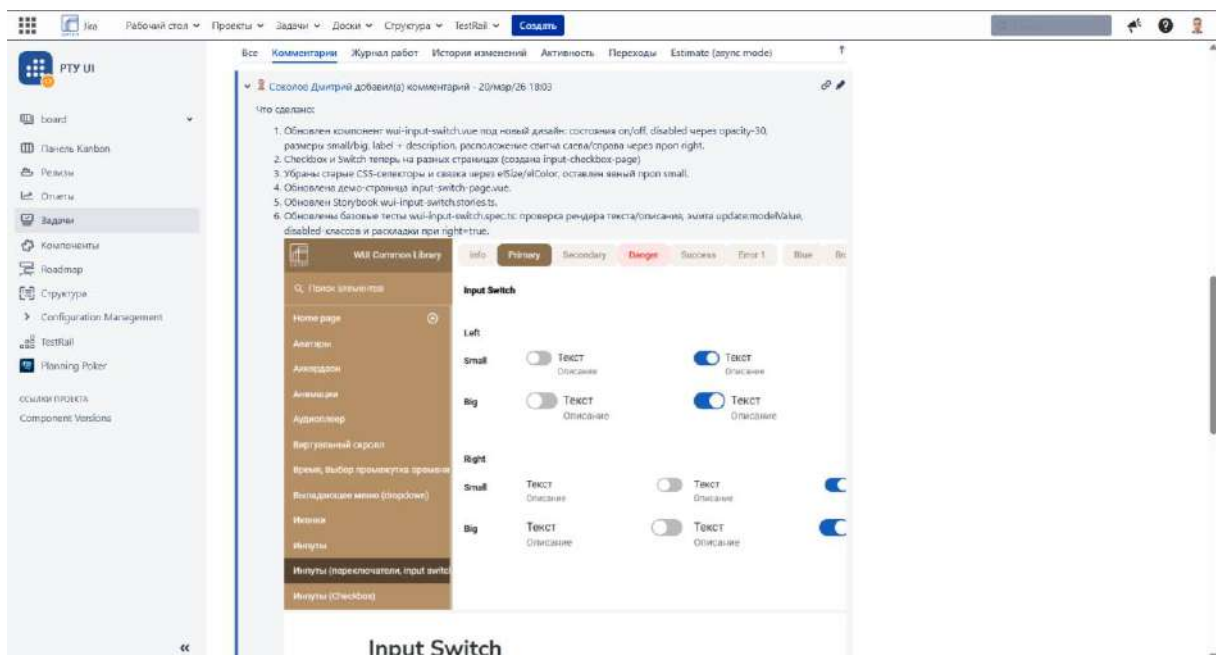


Рис. 3.6. Отчет в Jira по задаче WUI-4218

3.2.3 Обновление демонстрационных страниц и Storybook

После изменения компонента была обновлена демонстрационная страница `input-switch-page.vue`. Также `checkbox` и `switch` были разделены по разным страницам, чтобы каждый компонент демонстрировался отдельно и не смешивался с другим элементом интерфейса.

Отдельно был обновлен файл Storybook для компонента `wui-input-switch`. Storybook используется для изолированной демонстрации UI-компонентов, поэтому при изменении компонента важно обновлять его stories [15]. Это позволяет разработчикам и тестировщикам быстро увидеть все состояния компонента без запуска полного пользовательского сценария.

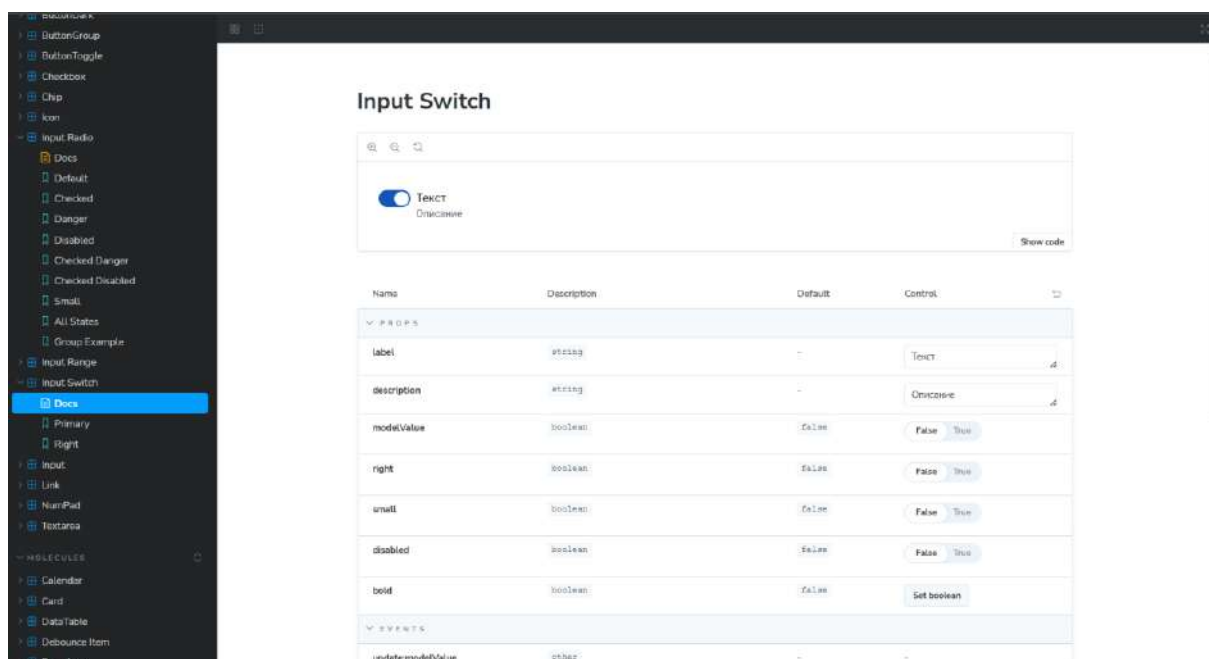


Рис. 3.7. Storybook по задаче WUI-4218

3.2.4 Написание и актуализация unit-тестов UI-компонентов

После доработки UI-компонента были обновлены базовые тесты `wui-input-switch.spec.ts` [19]. Тестирование было направлено на проверку основных сценариев работы компонента.

В тестах проверялись:

- корректный рендер текста и описания;
- срабатывание события `update:modelValue`;

- применение disabled-классов;
- корректная раскладка при параметре right=true.

Наличие таких тестов позволяет быстрее выявлять ошибки при дальнейших изменениях компонента. Это особенно важно для общей UI-библиотеки, так как один компонент может использоваться в нескольких продуктах и разделах приложения.

Результаты запуска теста приведены на рисунке 3.8.



Рис. 3.8. Запуск теста wui-input-switch.spec.ts

3.3 Доработка конфигурации запуска проекта в dev-режиме

Отдельным направлением работы стала задача WUI-4207, связанная с исправлением конфигурации запуска проекта в dev-режиме [18; 21; 22]. Данная задача относилась к техническим работам и была направлена на упрощение запуска проекта в разных окружениях.

3.3.1 Анализ проблемы хардкода backend-адресов

До выполнения доработки часть адресов backend-серверов была указана напрямую в конфигурационных файлах и скриптах запуска. Такой подход усложнял переключение между окружениями и мог приводить к ошибкам при сборке web-версии или Electron-приложения.

На этапе анализа были изучены файлы конфигурации проекта, логика формирования URL и места, где использовались адреса backend-серверов [18]. Было выявлено, что настройки необходимо вынести в переменные окружения, чтобы не редактировать конфигурацию вручную при каждом переключении стенда. На рисунке 3.9 приведена часть конфигурационного файла vite.config до исправления. Адреса backend сервисов скрыты по согласованию с руководителем практики от предприятия.



```
vite.config.ts
6 import VueDevTools from 'vite-plugin-vue-devtools'
7 import zipPack from 'vite-plugin-zip-pack'
8
9 import manifest from './manifest.json'
10 import { version } from './package.json'
11
12 const targets = {
13   dev: [REDACTED] 234
14   dev2: [REDACTED]
15   dev3: [REDACTED]
16   stage: [REDACTED] // 235
17   stage2: [REDACTED]
18   stage3: [REDACTED]
19   stage4: [REDACTED]
20   moscow: [REDACTED]
21   qa: 'http://localhost:8080'
22   rtuInter: [REDACTED]
23   rtum: 'http://localhost:8080'
24 }
25 // TODO не забыть про файл src\shared\url-helper\index.ts переменную DEV_SERVER
26 // https://vitejs.dev/config/
27 export default defineConfig(({ mode }) => {
28   const isDevelopment = /development/.test(mode)
29   const isStage = /stage/.test(mode)
30   const isProduction = !isDevelopment && !isStage
31
32   const target = targets.dev3
33
34   const isElectron = () => process.env.ELECTRON === 'true'
35
36   if (isElectron()) {
37     console.log('isElectron', isElectron())
38   }
```

Рис. 3.9. Конфигурационный файл до изменений

3.3.2 Вынесение параметров окружения в env-файлы

Для решения проблемы были добавлены отдельные конфигурации окружений, в том числе для development и stage. Адреса backend-серверов были вынесены в env-файлы.

Такой подход позволил отделить настройки окружения от основного кода приложения. В результате проект стало проще запускать в разных режимах без ручного изменения адресов в vite.config.ts, package.json или других файлах. Пример env файлов приведен на рисунке 3.10.

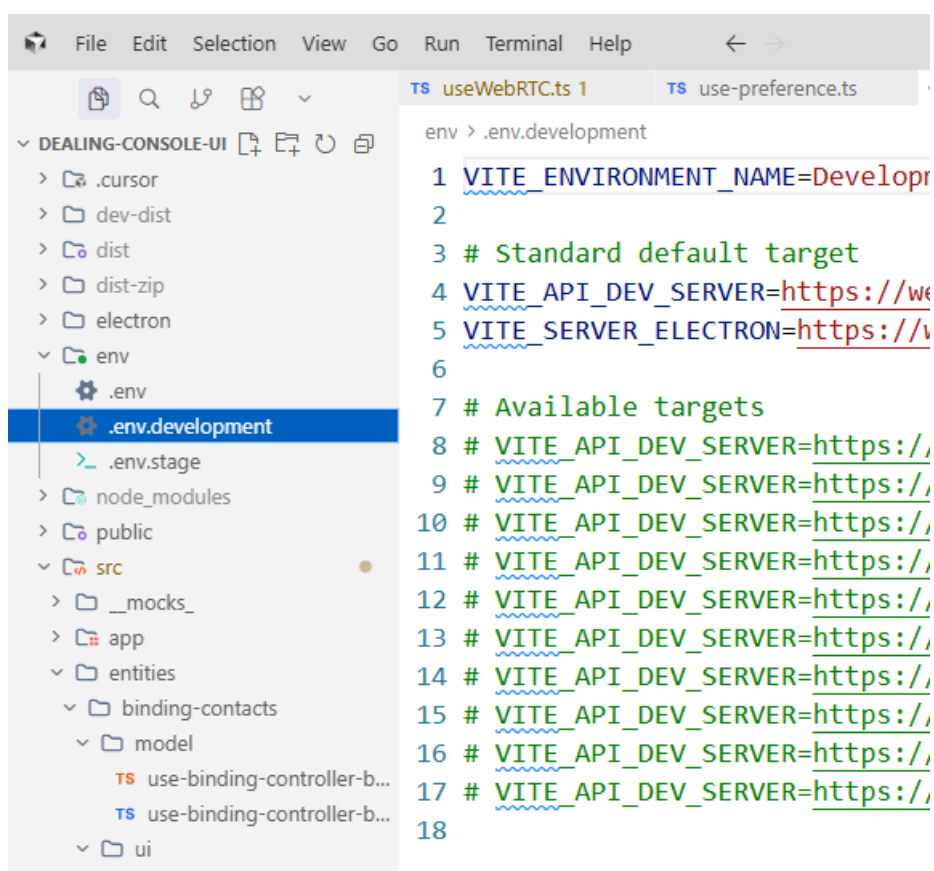


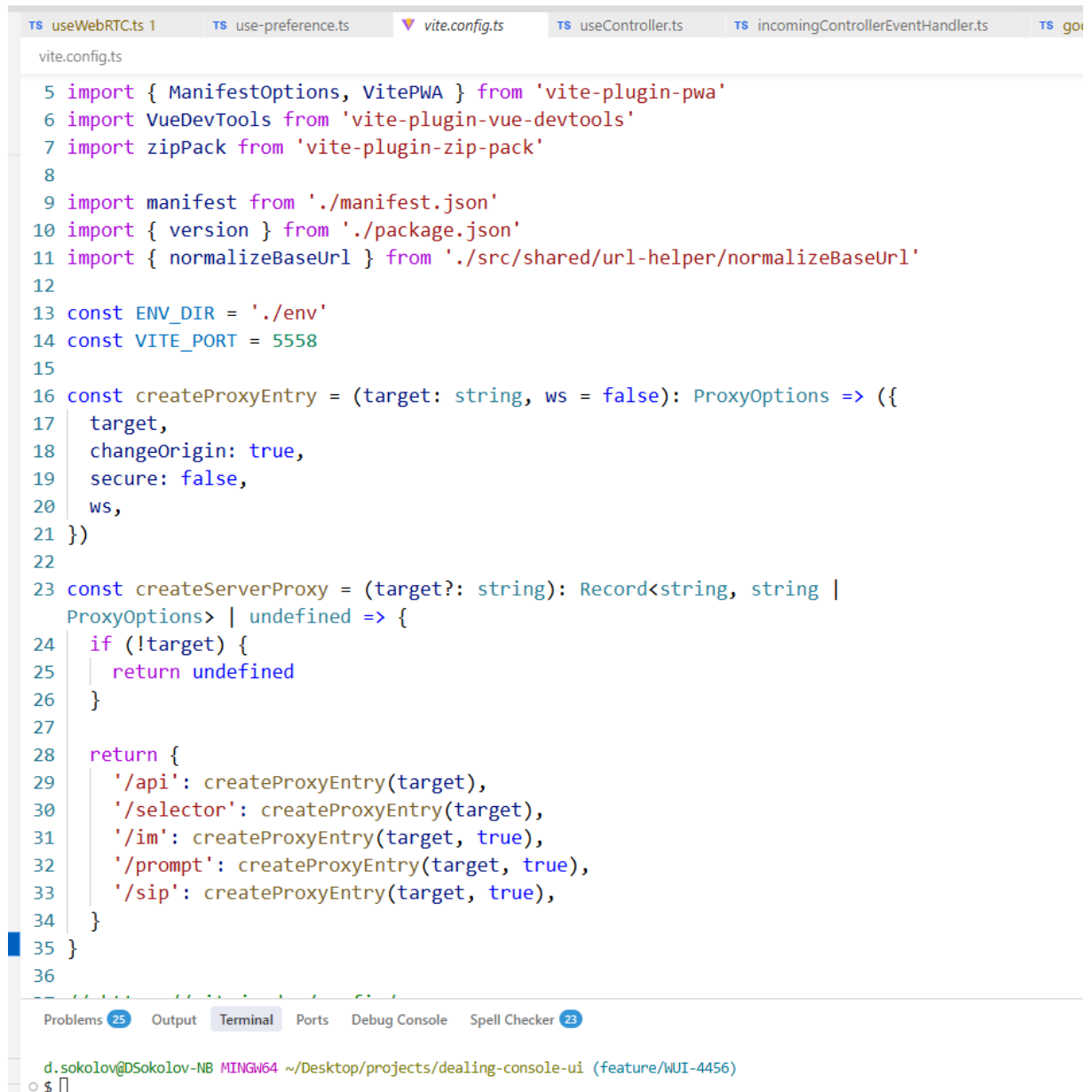
Рис. 3.10. Файлы env

3.3.3 Синхронизация формирования URL в dev-server и runtime

В рамках задачи была синхронизирована логика формирования URL между dev-server и runtime-частью приложения. Файл vite.config.ts был переведен на чтение переменных окружения через loadEnv [18].

Также была согласована логика формирования адресов в модуле `src/shared/url-helper/index.ts`. Это позволило использовать единый источник настроек как при локальном запуске, так и во время работы приложения.

Дополнительно из `package.json` были убраны жестко заданные адреса для Electron-сборки. Благодаря этому web- и Electron-версии стали использовать настройки из переменных окружения [11; 18]. Новый `vite.config` приведен на рисунке 3.11.



```
vite.config.ts

5 import { ManifestOptions, VitePWA } from 'vite-plugin-pwa'
6 import VueDevTools from 'vite-plugin-vue-devtools'
7 import zipPack from 'vite-plugin-zip-pack'
8
9 import manifest from './manifest.json'
10 import { version } from './package.json'
11 import { normalizeBaseUrl } from './src/shared/url-helper/normalizeBaseUrl'
12
13 const ENV_DIR = './env'
14 const VITE_PORT = 5558
15
16 const createProxyEntry = (target: string, ws = false): ProxyOptions => ({
17   target,
18   changeOrigin: true,
19   secure: false,
20   ws,
21 })
22
23 const createServerProxy = (target?: string): Record<string, string |
  ProxyOptions> | undefined => {
24   if (!target) {
25     return undefined
26   }
27
28   return {
29     '/api': createProxyEntry(target),
30     '/selector': createProxyEntry(target),
31     '/im': createProxyEntry(target, true),
32     '/prompt': createProxyEntry(target, true),
33     '/sip': createProxyEntry(target, true),
34   }
35 }
36
```

Рис. 3.11. Новый vite.config

3.3.4 Исправление формы авторизации

В рамках той же задачи были внесены небольшие изменения в форму авторизации. Логика состояния логина, пароля и флага отправки была вынесена в родительский компонент, а лишний `useForm` был удален.

Это упростило структуру компонента и сделало управление состоянием формы более прозрачным. В результате форма авторизации стала легче для сопровождения и дальнейшей доработки. Результат приведен на рисунке 3.12.



Рис. 3.12. Улучшенная форма авторизации

3.4 Реализация функциональности аппаратной индикации микрофона

Наиболее сложной практической задачей стала WUI-4278, связанная с реализацией световой индикации аппаратной кнопки микрофона. Эта задача относилась к функциональности дилингового пульта и была связана не только с интерфейсом, но и с аппаратными контроллерами.

Дилинговый пульт РТУ поддерживает работу с аппаратными модулями, включая Goose [2]. Goose-модуль содержит микрофон типа «гусиная шея» и аппаратные кнопки с LED-индикацией. Поэтому frontend должен синхронизировать состояние интерфейса, завешенных вызовов и физического устройства.

3.4.1 Анализ сценария работы аппаратной кнопки микрофона

Перед реализацией была проанализирована логика работы микрофона в дилинговом пульте. В продукте используются завешенные линии, при которых пользователь может одновременно слушать несколько вызовов и управлять тем, в какие линии передается звук с микрофона [2].

Основная задача заключалась в том, чтобы световая индикация аппаратной кнопки соответствовала фактическому состоянию микрофона. Если микрофон включен, кнопка должна подсвечиваться; если выключен - подсветка должна гаснуть.

Также нужно было учитывать, что состояние микрофона связано не только с одной карточкой вызова, но и с глобальным состоянием Goose-модуля. Интерфейс завешенных вызовов и UI состояние микрофона приведен на рисунке 3.13.



Рис. 3.13. Завешенные линии

3.4.2 Реализация глобального состояния goose-модулей

В ходе выполнения задачи состояние Goose-модулей было вынесено на глобальный уровень. Это позволило сделать единый источник состояния,

который используется как интерфейсом, так и логикой управления аппаратной подсветкой.

Итоговая слышимость для завешенной линии стала определяться по условию:

`gooseEnabled && line.micState`

Это означает, что пользователь будет слышен в конкретной линии только в том случае, если глобальное состояние Goose включено и у самой линии активирован микрофон.

Такой подход позволил разделить два уровня управления:

- глобальное состояние Goose-модуля;
- состояние микрофона конкретной завешенной линии [2].

Если Goose глобально выключен, пользователь не будет слышен ни в одной линии, даже если у отдельной карточки `micState = true`. Если Goose глобально включен, звук передается только в те линии, где микрофон включен локально. Часть кода данной задачи приведена на рисунке 3.14.

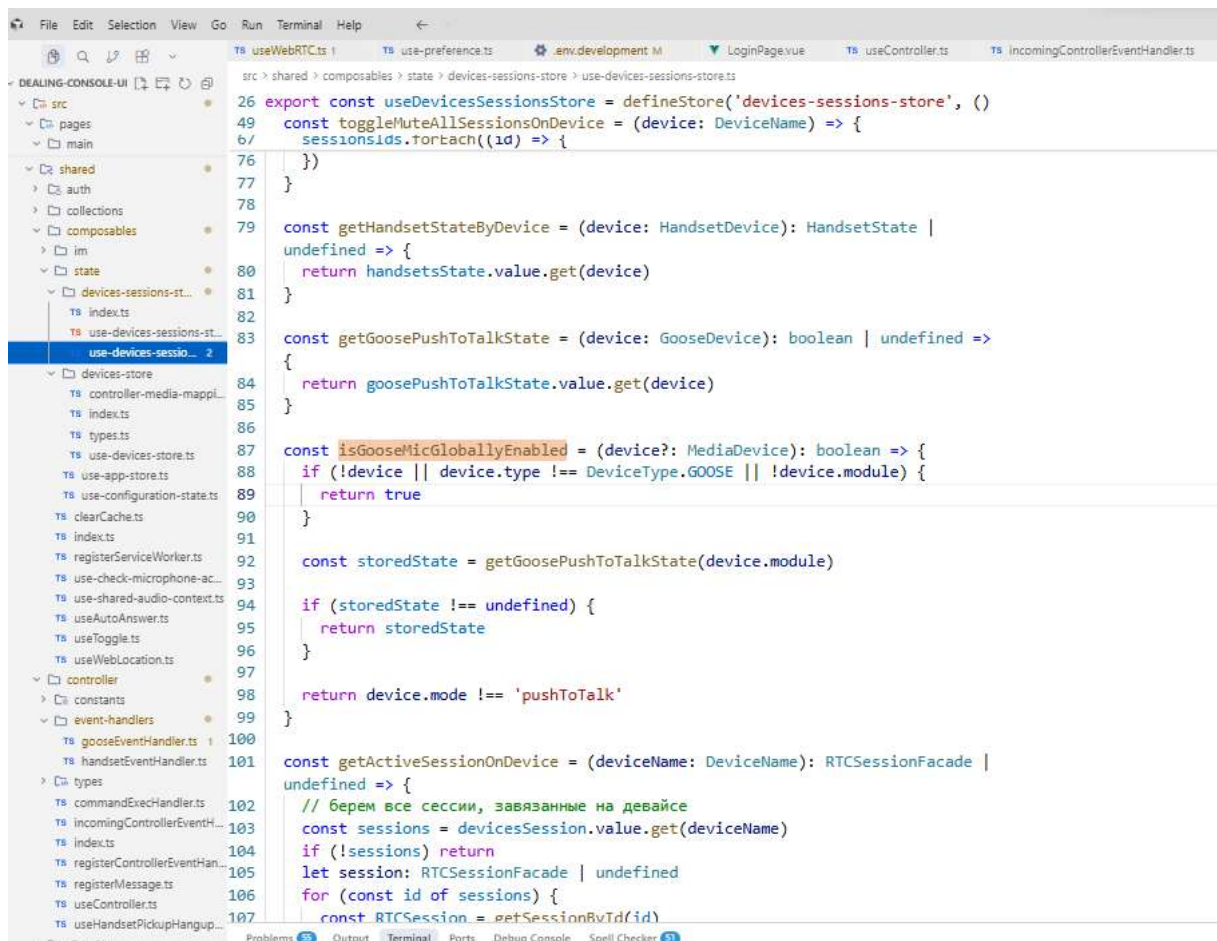


Рис. 3.14. Реализация глобального состояния микрофона

3.4.3 Синхронизация UI-кнопки mute и аппаратной индикации

После введения глобального состояния была синхронизирована UI-кнопка глобального mute и аппаратная кнопка микрофона. Обе кнопки стали читать и изменять одно и то же состояние.

Также была реализована логика, при которой новый входящий вызов, становясь завешенным, сразу получает текущее глобальное состояние Goose. Например, если Goose был выключен, новый разговор сразу стартует в состоянии muted.

Дополнительно был исправлен формат сообщений к контроллеру, а доступные модули стали подтягиваться автоматически с контроллера. Это упростило взаимодействие frontend-части с аппаратной частью приложения. UI кнопка приведена на рисунке 3.15, индикация кнопки KEY_SPEAK на рисунке 3.16.



Рис. 3.15. Глобальная UI кнопка микрофона для завешенных линий



Рис. 3.16. Световая индикация кнопки KEY_SPEAK

3.4.4 Особенности режимов stateful и push-to-talk

В задаче были учтены два режима работы микрофона: stateful и pushToTalk.

В режиме stateful аппаратная кнопка работает как переключатель. При первом нажатии микрофон включается, а подсветка загорается. При повторном нажатии микрофон выключается, а подсветка гаснет.

В режиме pushToTalk микрофон включается только на время удержания кнопки. После отпускания кнопки микрофон сразу выключается, а подсветка гаснет. Такой режим соответствует принципу «нажал - говоришь, отпустил - микрофон выключен».

Также была реализована логика, при которой при переключении Goose из режима stateful в pushToTalk устройство сразу переводится в выключенное состояние. Это необходимо, потому что режим pushToTalk требует удержания кнопки для передачи звука.

В результате выполнения задачи была обеспечена более точная синхронизация между состоянием интерфейса, завешенными вызовами и аппаратной кнопкой микрофона. Пример интерфейса приведен на рисунке 3.17.

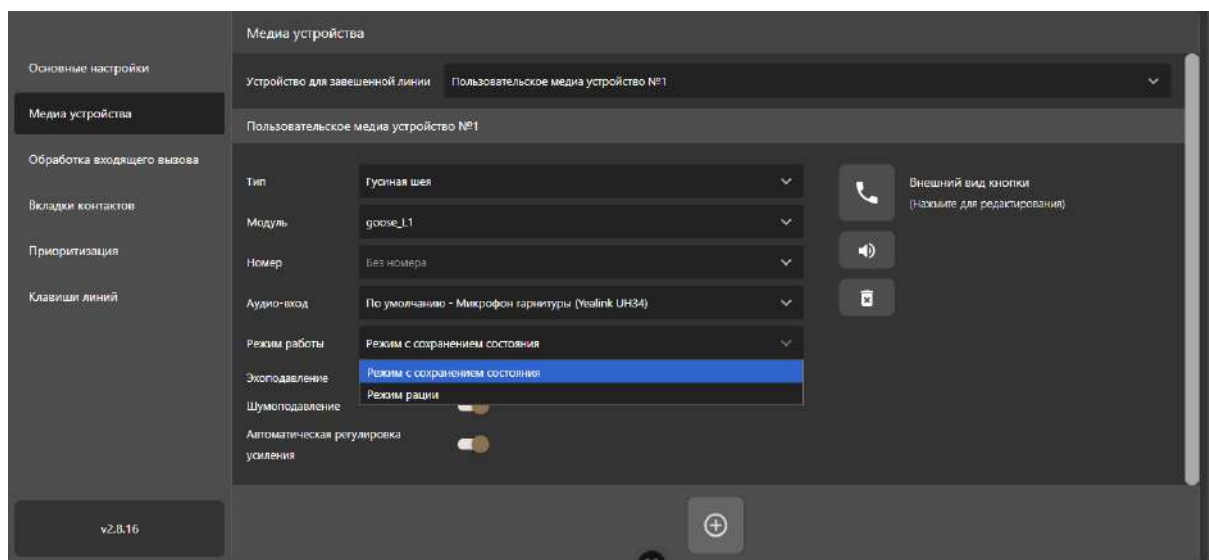


Рис. 3.17. Реализация режимов работы кнопки KEY_SPEAK

Заключение

В ходе прохождения проектно-технологической практики в ООО «САТЕЛ» были закреплены и расширены знания, полученные в процессе обучения по направлению «Прикладная информатика». Практика была связана с выполнением задач frontend-разработчика в рамках сопровождения и развития программного продукта «Дилинговый пульт РТУ».

В период практики была изучена деятельность профильной организации, основные направления разработки программных продуктов, особенности платформы РТУ и специфика дилингового пульта как специализированного интерфейса для работы с телефонными вызовами, завешенными линиями и аппаратными контроллерами.

В процессе выполнения практических задач были применены современные технологии frontend-разработки: Vue 3, TypeScript, Vite, Pinia, Tailwind CSS, Storybook, Git, GitLab, Jira и Figma. Были изучены рабочие процессы команды разработки: анализ задачи, работа с макетами, создание ветки, внесение изменений, локальная проверка, оформление merge request, прохождение code review и передача задачи на тестирование.

Поставленные цели и задачи практики были выполнены. Полученный опыт позволил лучше понять полный цикл разработки программного обеспечения в реальной организации и повысить уровень профессиональной подготовки в области frontend-разработки.

Библиографический список

1. Онбординг Frontend: полное руководство по первому дню и рабочему процессу : внутренний электронный документ / ООО «САТЕЛ». – Нижний Новгород, 2026. – 6 с. – Текст : электронный.
2. Спецификация проекта «Дилинговый пульт РТУ» : версия 1.4 : внутренний электронный документ / ООО «САТЕЛ». – Нижний Новгород, 2026. – Текст : электронный.
3. Диков, А. В. Клиентские технологии веб-программирования: JavaScript и DOM : учебное пособие для вузов / А. В. Диков. – 2-е изд., стер. – Санкт-Петербург : Лань, 2026. – 124 с. – ISBN 978-5-507-54803-3. – Текст : электронный // ЭБС «Лань» : [сайт]. – URL: <https://e.lanbook.com/book/510752> (дата обращения: 29.04.2026). – Режим доступа: для авториз. пользователей.
4. Дронов, В. А. JavaScript. 32 урока для начинающих / В. А. Дронов. – Санкт-Петербург : БХВ-Петербург, 2024. – 576 с. – ISBN 978-5-9775-1942-7. – Текст : электронный // ЭБС «Ibooks.ru» : [сайт]. – URL: <https://ibooks.ru/bookshelf/396454/reading> (дата обращения: 29.04.2026). – Режим доступа: по подписке.
5. Игнатьев, А. В. Тестирование программного обеспечения / А. В. Игнатьев. – 2-е изд., стер. – Санкт-Петербург : Лань, 2022. – 56 с. – ISBN 978-5-8114-9936-6. – Текст : электронный // ЭБС «Лань» : [сайт]. – URL: <https://e.lanbook.com/book/201188> (дата обращения: 29.04.2026). – Режим доступа: для авториз. пользователей.
6. Коваленко, Т. А. Проектирование пользовательского интерфейса : учебник / Т. А. Коваленко, А. Л. Золкин. – Самара : ПГУТИ, 2024. – 154 с. – ISBN 978-5-507-53055-7. – Текст : электронный // ЭБС «Лань» : [сайт]. – URL: <https://e.lanbook.com/book/463541> (дата обращения: 29.04.2026). – Режим доступа: для авториз. пользователей.
7. Прохоренок, Н. А. JavaScript и Node.js для веб-разработчиков / Н. А. Прохоренок, В. А. Дронов. – Санкт-Петербург : БХВ-Петербург, 2022. –

768 с. – ISBN 978-5-9775-6847-0. – Текст : электронный // ЭБС «Ibooks.ru» : [сайт]. – URL: <https://www.ibooks.ru/bookshelf/385754/reading> (дата обращения: 29.04.2026). – Режим доступа: по подписке.

8. Янцев, В. В. JavaScript. Креативное программирование / В. В. Янцев. – Санкт-Петербург : Лань, 2023. – 232 с. – ISBN 978-5-507-45405-1. – Текст : электронный // ЭБС «Лань» : [сайт]. – URL: <https://e.lanbook.com/book/303998> (дата обращения: 29.04.2026). – Режим доступа: для авториз. пользователей.

9. РТУ : платформа унифицированных коммуникаций. – Текст : электронный // ООО «САТЕЛ» : официальный сайт. – URL: <https://satel.ru/development/rtu-platforma-unifitsirovannykh-kommunikatsiy/> (дата обращения: 29.04.2026).

10. О компании. – Текст : электронный // ООО «САТЕЛ» : официальный сайт. – URL: <https://satel.ru/company/about/> (дата обращения: 29.04.2026).

11. Introduction. – Текст : электронный // Electron : [сайт]. – URL: <https://www.electronjs.org/docs/latest/> (дата обращения: 29.04.2026).

12. Figma Design. – Текст : электронный // Figma : [сайт]. – URL: <https://www.figma.com/design/> (дата обращения: 29.04.2026).

13. Git : [сайт]. – Текст : электронный. – URL: <https://git-scm.com/> (дата обращения: 29.04.2026).

14. Getting Started. – Текст : электронный // Pinia : [сайт]. – URL: <https://pinia.vuejs.org/getting-started.html> (дата обращения: 29.04.2026).

15. Get started with Storybook. – Текст : электронный // Storybook : [сайт]. – URL: <https://storybook.js.org/docs> (дата обращения: 29.04.2026).

16. Installing Tailwind CSS with Vite. – Текст : электронный // Tailwind CSS : [сайт]. – URL: <https://tailwindcss.com/docs> (дата обращения: 29.04.2026).

17. TypeScript : [сайт]. – Текст : электронный. – URL: <https://www.typescriptlang.org/> (дата обращения: 29.04.2026).

18. Getting Started. – Текст : электронный // Vite : [сайт]. – URL: <https://vite.dev/guide/> (дата обращения: 29.04.2026).
19. Getting Started. – Текст : электронный // Vitest : [сайт]. – URL: <https://vitest.dev/guide/> (дата обращения: 29.04.2026).
20. Introduction. – Текст : электронный // Vue.js : [сайт]. – URL: <https://vuejs.org/guide/introduction> (дата обращения: 29.04.2026).
21. GitLab Docs : [сайт]. – Текст : электронный. – URL: <https://docs.gitlab.com/> (дата обращения: 18.05.2026).
22. Jira Cloud support. – Текст : электронный // Atlassian Support : [сайт]. – URL: <https://support.atlassian.com/jira-software-cloud/> (дата обращения: 18.05.2026).
23. Getting Started. – Текст : электронный // Vue Router : [сайт]. – URL: <https://router.vuejs.org/guide/> (дата обращения: 18.05.2026).

**Федеральное государственное бюджетное образовательное учреждение
высшего образования
«РОССИЙСКАЯ АКАДЕМИЯ НАРОДНОГО ХОЗЯЙСТВА И
ГОСУДАРСТВЕННОЙ СЛУЖБЫ
при ПРЕЗИДЕНТЕ РОССИЙСКОЙ ФЕДЕРАЦИИ»**

НИЖЕГОРОДСКИЙ ИНСТИТУТ УПРАВЛЕНИЯ – ФИЛИАЛ

Факультет управления

Кафедра информатики и информационных технологий

Направление подготовки: 09.03.03 Прикладная информатика

Образовательная программа: Корпоративные информационные системы
управления

**ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ
по технологической (проектно-технологической) практике**

Обучающегося 3 курса, учебной группы № Ик-731

Соколова Дмитрия Александровича

(Ф.И.О. обучающегося полностью)

Место прохождения практики: ООО «САТЕЛ ПрО», Отдел проектных разработок
Департамента унифицированных коммуникаций Основного подразделения,

адрес организации: г. Москва, ул. Щепкина, д. 28

(указывается полное наименование структурного подразделения НИУ – филиала РАНХиГС / профильной организации и её структурного подразделения, а также их фактический адрес)

Срок прохождения практики с «27» апреля 2026 г. по «26» мая 2026 г.

Содержание индивидуального задания:

- литературные источники по разрабатываемой теме с целью их использования при выполнении бакалаврской выпускной работы;
- методы исследования и проведения работ, анализа и обработки данных;
- информационные технологии в профессиональной деятельности;
- программные продукты, относящиеся к профессиональной сфере;
- требования к оформлению научно-технической документации;
- порядок внедрения результатов разработок.

Планируемые результаты:

- закрепление, углубление и расширение знаний, полученных в ходе обучения.
- приобретение новых, закрепление, углубление и расширение имеющихся профессиональных умений и навыков.
- получение реального опыта полноценной профессиональной деятельности.

СОГЛАСОВАНО¹

С.М. Митричев

И.О. Фамилия руководителя по практической
подготовке от
профильной организации

«27» апреля 2026 г.

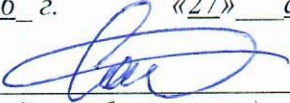
УТВЕРЖДАЮ

Н.М. Трубилов

И.О. Фамилия руководителя по практической
подготовке от Института

«27» апреля 2026 г.

Задание принято к исполнению:


(подпись обучающегося)

«27» апреля 2026 г.

¹ при прохождении практики в профильной организации

ОТЗЫВ
о работе обучающегося в период прохождения практики

Обучающийся Соколов Дмитрий Александрович
(Ф.И.О обучающегося)

Нижегородского института управления – филиала РАНХиГС

проходил технологическую (проектно-технологическую) практику
(вид практики)

в период с 27 апреля 2026 г. по 26 мая 2026 г.

в ООО «САТЕЛ ПрО» Отдел проектных разработок Департамента унифицированных коммуникаций Основного подразделения

(наименование профильной организации с указанием структурного подразделения)

в качестве практиканта.

Обучающийся был ознакомлен с условиями прохождения практики в организации, после чего был допущен к выполнению определенных индивидуальным заданием видов работ, связанных с будущей профессиональной деятельностью.

В период прохождения практики обучающийся проявил практические навыки, активность, дисциплину. Следует отметить качество и достаточность собранного материала для отчета. Отчет по практике соответствует индивидуальному заданию

(практические навыки, активность, дисциплина, помощь профильной организации, качество и достаточность собранного материала для отчета и выполненных работ, поощрения и т.п.)

К должностным обязанностям и поставленным задачам в соответствии с индивидуальным заданием практикант относился добросовестно, проявляя интерес к работе. Порученные задания выполнил в полном объеме в установленные программой практики сроки.

Считаю, что по итогам практики обучающийся может быть допущен к защите отчета по практике.

Руководитель отдела разработки (подпись)
(должность руководителя по практической подготовке от профильной организации)

Митяев С.М.
(И.О. Фамилия)



«26» мая 2026г.



АНТИПЛАГИАТ
ОБНАРУЖЕНИЕ ЗАИМСТВОВАНИЙ

СПРАВКА

о результатах проверки текстового документа
на наличие заимствований

Федеральное государственное бюджетное
образовательное учреждение высшего
образования "Российская академия народного
хозяйства и государственной службы при
Президенте Российской Федерации"

ПРОВЕРКА ВЫПОЛНЕНА В СИСТЕМЕ АНТИПЛАГИАТ.VUZ

Автор работы: Соколов Дмитрий Александрович

Самоцитирование
рассчитано для:

Название работы: 2 Соколов Д.А. Ик-731 (проектно-технологическая практика) ООО 'Сател' (исправлено).docx

Тип работы: Курсовая работа

Подразделение:

РЕЗУЛЬТАТЫ

■ ОТЧЕТ О ПРОВЕРКЕ КОРРЕКТИРОВАЛСЯ: НИЖЕ ПРЕДСТАВЛЕНЫ РЕЗУЛЬТАТЫ ПРОВЕРКИ ДО КОРРЕКТИРОВКИ

СОВПАДЕНИЯ	3.06%	СОВПАДЕНИЯ	3.06%
ОРИГИНАЛЬНОСТЬ	96.94%	ОРИГИНАЛЬНОСТЬ	96.94%
ЦИТИРОВАНИЯ	0%	ЦИТИРОВАНИЯ	0%
САМОЦИТИРОВАНИЯ	0%	САМОЦИТИРОВАНИЯ	0%
ИИ-КОНТЕНТ	0%		

ДАТА И ВРЕМЯ КОРРЕКТИРОВКИ: 19.05.2026 16:06

ДАТА ПОСЛЕДНЕЙ ПРОВЕРКИ: 19.05.2026

Структура
документа:

Проверенные разделы: основная часть с.9-37, содержание с.1-2, введение с.3-9, выводы с.38

Модули поиска:

Профессиональная лексика. Медицина; Шаблонные фразы; Публикации РГБ; Сводная коллекция научных работ Беларуси; Цитирование; Перефразирование по коллекции издательства Wiley; Издательство Wiley; ИПС Адилет; Коллекция НБУ; PubMed; СПС ГАРАНТ: аналитика; Коллекция открытых публикаций международных издательств; Профессиональная лексика. АПК и биотех; Профессиональная лексика. Юриспруденция; СМИ России и СНГ; IEEE; Патенты СССР, РФ, СНГ; Переводные заимствования по коллекции Интернет в русском сегменте; Публикации eLIBRARY (переводы и перефразирование); Перефразирование по Коллекции открытых публикаций международных издательств; Публикации eLIBRARY; Кольцо вузов; Переводные заимствования; Перефразирование по коллекции IEEE; Перефразирование по базе публикаций...

Работу проверил:

Соколов Дмитрий Александрович

ФИО проверяющего

Дата подписи:

19.05.2026

[Подпись]

Подпись проверяющего



Чтобы убедиться
в подлинности справки, используйте QR-код,
который содержит ссылку на отчет.

Ответ на вопрос, является ли обнаруженное заимствование
корректным, система оставляет на усмотрение проверяющего.
Предоставленная информация не подлежит использованию
в коммерческих целях.